

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 周禹驰 学号 2024K8009929027 专业 计算机科学与技术
实验项目编号 1 实验名称 寄存器堆和算术逻辑单元

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下 (注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支 (具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、逻辑电路结构与仿真波形的截图及说明 (比如 Verilog HDL 关键代码段 {包含注释} 及其对应的逻辑电路结构图 {自行画图, 推荐用 PPT 画逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX. 中}、相应信号的仿真波形和信号变化的说明等)

• 寄存器堆设计。

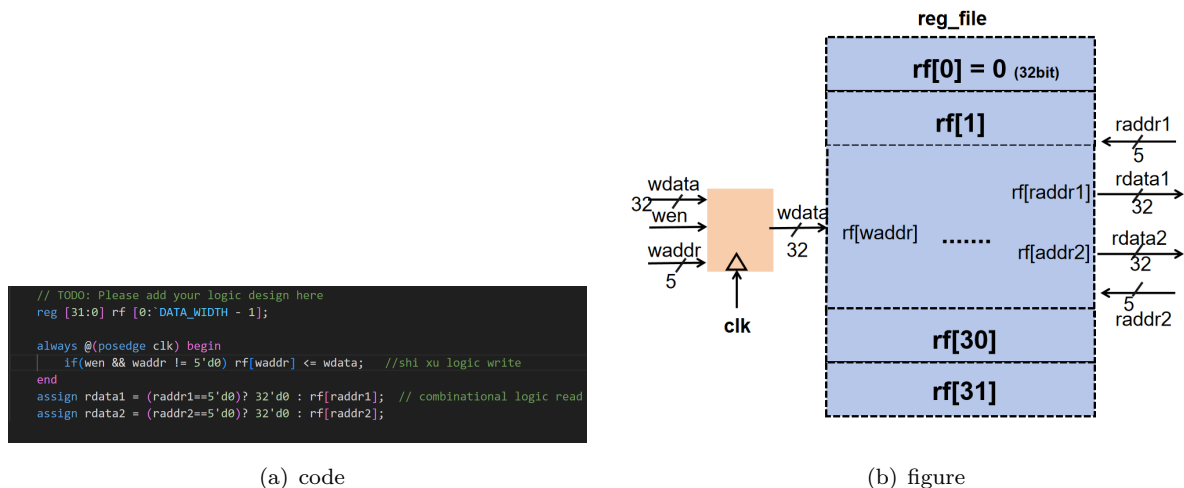


图 1: 寄存器堆代码和电路简图

- ALU 设计。
- 其他关键模块设计。请输入你的实验报告内容。
如果一段话写不下, 可以再这里继续新的段落。

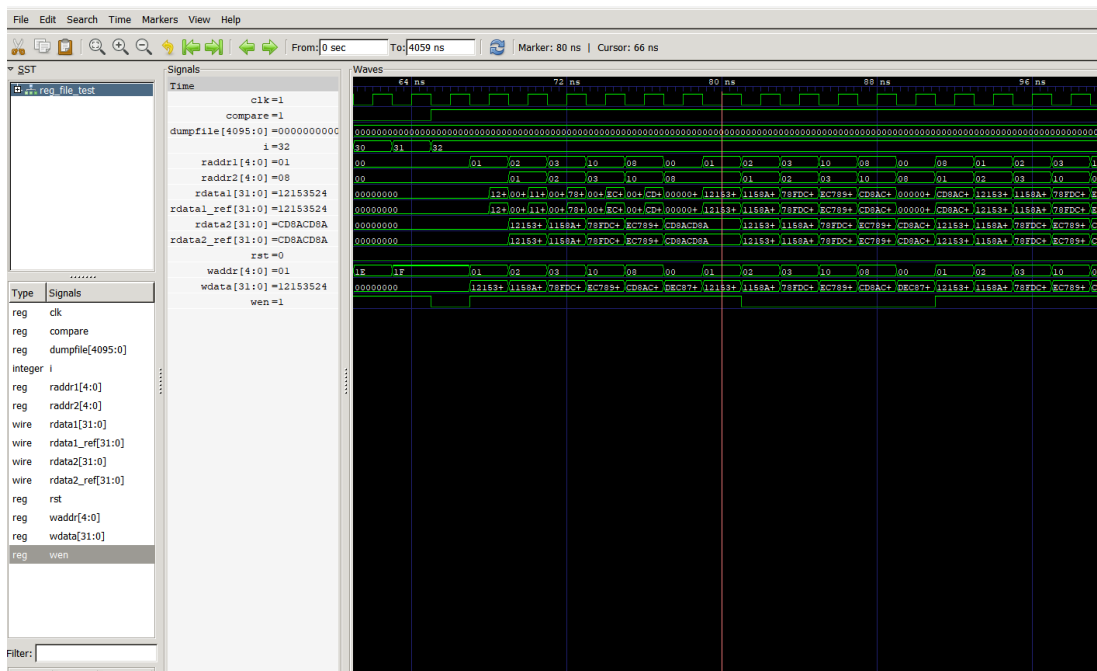


图 2: reg_file 波形图

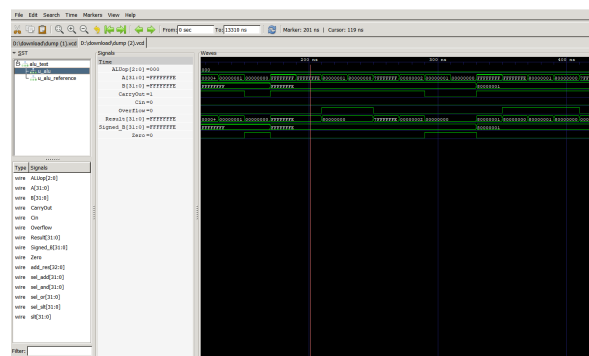
```
// TODO: Please add your logic design here
wire Cin;
wire [DATA_WIDTH-0] add_res;
wire [DATA_WIDTH-1:0] Signed_B; //the value of "B" that join caculate
wire [DATA_WIDTH-1:0] s1t;
wire [DATA_WIDTH-1:0] sel_and; //signs that enable multi_select
wire [DATA_WIDTH-1:0] sel_or;
wire [DATA_WIDTH-1:0] sel_add;
wire [DATA_WIDTH-1:0] sel_slt;

assign Signed_B = ALUop[2]? ~B:B; //110 --sub --> A+(-B+1), 010 -- add --> A+B
assign Cin = ALUop[2];
assign add_res = A + Signed_B + Cin;
assign CarryOut = (ALUop[2]? ~add_res[DATA_WIDTH] : add_res[DATA_WIDTH]);
assign Overflow = (A[DATA_WIDTH-1] == Signed_B[DATA_WIDTH-1]) &&
(A[DATA_WIDTH-1] ^ add_res[DATA_WIDTH-1]);
assign s1t = {31'd0, (add_res[DATA_WIDTH-1] ^ Overflow)};

assign sel_and = {32((ALUop == 3'b000))};
assign sel_or = {32((ALUop == 3'b001))};
assign sel_add = {32((~ALUop[0] & ALUop[1]))};
assign sel_slt = {32((ALUop == 3'b111))};

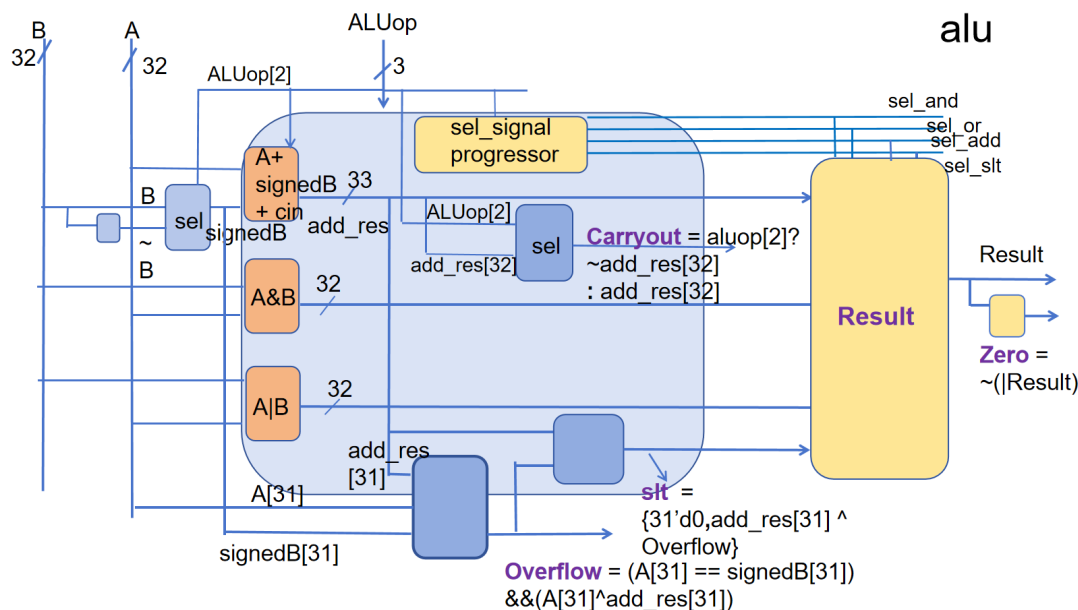
assign Result = (sel_and & (A&B)) | (sel_or & (A|B)) | //use mul_sel
(sel_add & add_res[DATA_WIDTH-1:0]) | (sel_slt & s1t);
assign Zero = ~(Result);
```

(a) code



(b) figure

图 3: alu 代码和波形简图



二、实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug,逻辑仿真和 FPGA 调试过程中的难点等)

- **寄存器堆设计问题。**对于实现 0 号寄存器读出 0 的实现,我采用了选择器的写法,先判断读地址是否为 0,是则读 0,不是则正常读
- **ALU 设计问题。**

1. **carryout 信号的处理:** 一开始, 我将 carryout 的公式写为: $carryout = address[DATAWIDTH]$; 但跑本地测试的时候遇到报错, 自己手推了一下测试数据, 发现是自己忽略了 carryout 既是进位又是借位的问题, 当执行加法的时候, carryout 是进位逻辑, 是相加结果的最高位, 即 $cout[32]$, 执行减法是借位逻辑, 是 $\neg cout[32]$, 因为减法是满足 $A - B = A + (2^{31} - B) = 2^{31} + (A - B)$, 当 $A < B$ 需要借位, 此时结果 $< 2^{31}$, 因此 $cout[32] = 0$, 所以发生借位应该把最高位取反

2. 关于 **result** 的处理: 一开始计划使用多个二元选择器来实现多路选择, 但是这样在实际搭建电路的时候延时会高, 为了一次性选择出结果, 创建了 4 个选择信号 *seland*, *selor*, *seland*, *selslt*, 最后使用简单的与或逻辑实现了四路选择

3. **slt 和四个选择信号的处理:** 一开始对于 slt 和四个选择信号的产生, 我都是使用选择器来实现的, 比如 $slt = addres[31] \wedge Overflow? 32'd1 : 32'd0$; 以及 $seland = (aluop == 3'b000)? 32'hFFFFFFF : 32'd0$; 但是这样也消耗了不必要的资源, 在助教的提示下, 我采用了位扩展的方式解决了这个问题, 将重复的位使用位扩展, 然后与有效信号位连接

- **其他关键模块设计问题。**请输入你的实验报告内容。
如果一段话写不下,可以再这里继续新的段落。

三、实验所耗时间

在课后,你花费了大约 4 小时完成此次实验。